Task 1 – Understand the Problem and Identify the Data

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K4 – Analyse

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task01)

1. Inputs and outputs

The clerk records one entry every time a farmer delivers milk (twice a day). The program must turn these raw readings into money, so every value is classed as an input (typed or read), a derived value (calculated) or an output (printed or paid).

Value	Kind	Type	Why this type
farmer ID (e.g. F001)	input	str	An identifier is never used in arithmetic; leading zeros and letters must be kept, which an int would lose.
shift (M / E)	input	str	A label with two values; only compared, never added.
litres	input	float / Decimal	Fractional (8.5 L); measured quantity.
fat %, SNF %	input	float / Decimal	Fractional (3.5, 8.7); must be compared with 3.5 and 8.5 exactly.
fat steps, SNF steps	derived	int	A count of 0.1 % steps is always a whole number (7, −4).
rate per litre	output	money (2 decimals)	Rupees and paise; has to match the bank to the paisa.
amount for one entry	output	money (2 decimals)	litres × rate, rounded to whole paise.
ten-day total	output	money (2 decimals)	Sum of the ten days' amounts for one farmer; this is what is transferred.
number of entries	derived	int	Counts entries (about 20 per farmer in ten days).

2. Values that never change during a run (named constants)

A constant is any value that is part of the co-operative's rules rather than part of one delivery. Naming them once is the direct answer to the volunteer's problem of rates being typed in several places.

Constant	Value	Reason it is a constant
BASE_RATE	30.00	Price of standard milk; changes only in a union revision, never during a run.
FAT_STEP_VALUE	0.40	Rs added/subtracted per 0.1 % of fat from the standard.
SNF_STEP_VALUE	0.30	Rs added/subtracted per 0.1 % of SNF from the standard.
STD_FAT, STD_SNF	3.5, 8.5	The quality at which the base rate applies.
STEP	0.1	Size of one step in per cent.
MIN_FAT, MAX_FAT	2.0, 7.0	Valid range of fat given in the problem.
PAYMENT_DAYS	10	Payment cycle (used when totalling).

3. Whole numbers, fractional numbers and text

- **Whole numbers (int):** number of steps, number of entries, number of days, and – if we choose integer paise – the money itself.
- **Fractional numbers:** litres, fat %, SNF %, the rates and the amounts. Money is fractional only because rupees have paise; this is why Task 2 studies float versus Decimal.
- **Text (str):** farmer ID, shift, and everything the clerk types, because input() always returns a string and conversion is the program's job (Task 5).

4. Rules every correct payment must satisfy

Rule	Justification
litres > 0 (and not absurdly large, e.g. ≤ 200 per entry)	A negative or zero quantity would give a negative or zero payment; a typing slip such as 1000 instead of 10.0 would be paid silently.
2.0 ≤ fat ≤ 7.0	Given in the problem; outside this range the milk is either adulterated or the analyser/clerk has made an error.
6.0 ≤ SNF ≤ 10.0 (assumption)	Not stated in the problem, but an SNF of 0 or 90 is clearly an entry error; the limits are named constants so the union can change them.
fat and SNF must be numbers	A text value ("eight", "8,5") cannot be used in the step formula.
rate > 0 and amount = round(litres × rate, 2)	The computed rate must never become zero or negative; the amount must be an exact number of paise.
farmer is paid the sum of already-rounded entry amounts	Matches how the bank statement is built (each credit is a whole number of paise), so totals agree to the paisa.

5. Assumptions

- Steps are counted by rounding to the nearest whole 0.1 % (readings are given to one decimal).
- Rounding of a half paisa is *half up* (Rs 1.505 becomes Rs 1.51), as a bank does.
- One entry = one farmer, one shift; ten days = about 20 entries per farmer.
- SNF limits (6.0–10.0) and the 200 L per-entry cap are my own sanity limits and are easy to change.

Alternative Concept and Complexity Analysis

Alternative: keep every entry in three separate names (litres1, fat1, snf1, litres2, …). For 300 farmers this means 900 separate assignments typed by hand, and a calculation such as `total = litres1*rate(fat1,snf1) + litres2*rate(fat2,snf2) + …` with 300 terms. A Python loop cannot visit variable names, so the code must be rewritten whenever the number of farmers changes. Even the best imitation of this idea – a dictionary with keys "litres1", "fat1" … – needs a string to be built for every access. I measured it:

Both designs (excerpt of storage_compare.py)

```
import random, sys, time

def make(n):
    rnd = random.Random(1)
    return [(round(rnd.uniform(2, 25), 1), rnd.randint(25, 65) / 10, rnd.randint(78, 92) / 10) for _ in range(n)]

def rate(fat, snf):
    return 30.0 + round((fat - 3.5) / 0.1) * 0.40 + round((snf - 8.5) / 0.1) * 0.30

def total_tuples(entries):                    # design B: one tuple per entry
    t = 0.0
    for litres, fat, snf in entries:
        t += litres * rate(fat, snf)
    return t

def total_named(ns, n):                       # design A: litres1, fat1, snf1 ... as separate names
    t = 0.0
    for i in range(1, n + 1):
        t += ns[f"litres{i}"] * rate(ns[f"fat{i}"], ns[f"snf{i}"])
    return t
```

Output

```
n=   300  tuples    0.12 ms  named    0.30 ms  container       21 KB vs       67 KB (names)  same total: True
n=  6000  tuples    2.39 ms  named    7.05 ms  container      427 KB vs     1264 KB (names)  same total: True
n=150000  tuples   59.67 ms  named  276.69 ms  container    10628 KB vs    37105 KB (names)  same total: True
lines of code to declare 300 farmers with separate names: 900 assignments (3 x 300)
```

	Design A: separate names	Design B: one tuple per entry in a list
Time to process n entries	O(n) but with a larger constant (string building + dict lookup, ≈ 4–5× slower here); code length is O(n) lines	O(n) with a loop of 3 lines; code length O(1)
Space	O(n) variables plus their name strings (≈ 3.5× more memory here)	O(n) – one small tuple per entry
Scaling 300 → 6,000 → 150,000	Source file grows with n; impossible to type	Same program; only the data grows

Conclusion: both are O(n) in running time, but design B is O(1) in program text and has a lower constant; design A’s program grows with the data, so it does not scale beyond a handful of farmers.

Evidence

The tables above give the inputs, outputs, constants, types, rules and assumptions, each with its reason; the measurement shows why the list/tuple design is chosen. Source: Task01/storage_compare.py.